Ders 12 Çalışma Özeti

Ders 12 Çalışma Özeti

Genel Konular

- Karakter dizileri (string) ve karakter problemleri
  - Karakter dizisinin her gözünde bir karakter saklanır. Karakter sabitleri tek tırnakla gösterilir: `'3'`, `'0'`, `'a'`.
  - Karakter saklayan değişkenler hiçbir zaman tek tırnakla yazılmaz. Tek tırnak yalnızca sabitler içindir.
  - Bu derste karakter sorularında kelimenin uzunluğu (N) ayrıca okunuyor. C'de buna gerek kalmayacak: okuma tamamlandığında sistem karakter grubunun sonuna otomatik olarak null karakteri (değeri 0) koyar. Dizinin sonu bu karakterden anlaşılır.
  - Bellekte her bilgi sayısal olarak saklanır. Karakterler de ASCII tablosundaki sayısal karşılıklarıyla saklanır. Bu yüzden karakterlerle toplama ve çıkarma yapılabilir.
- Örnek: rakam karakterlerinden sayı oluşturma (önceki dersten kalan soru)
  - Soru: Rakamlardan oluşan bir karakter dizisi verildiğinde, bu rakamların oluşturduğu sayıyı tek bir tam sayıda saklayan algoritma. Örnek: `'3' '0' '5'` → 305.
  - Karakterin sayısal değeri: `rakamlar[I] - '0'`. Örneğin `'3' - '0' = 3`. ASCII değerlerini bilmeye gerek yoktur, çünkü iki rakam karakteri arasındaki fark her zaman rakamların farkıdır.
  - Birinci çözüm (iki iç içe döngü):
    - `toplam = 0`. Dış döngü `I ← 1 || 1 || N`. Her turda `carpan = 1` yapılır.
    - İç döngü `J ← 1 || 1 || N−I` ile `carpan = carpan * 10` hesaplanır. Böylece I. rakamın basamak değeri 10^{N-I} bulunur.
    - `toplam = toplam + (rakamlar[I] − '0') * carpan`. Döngülerden sonra `toplam` yazdırılır.
  - İkinci çözüm (tek döngü, ekrandan, 00:43:20):
    - `toplam = 0`, `carpan = 1`. Döngü sağdan sola gider: `I ← N || −1 || 1`.
    - Döngü içinde: `sayi = rakamlar[I] − '0'`, `toplam = toplam + sayi * carpan`, `carpan = carpan * 10`.
    - Döngüden sonra `toplam` yazdırılır.
  - İkinci çözüm daha verimlidir. Çarpan her turda yalnızca bir kez 10 ile çarpılır. Birinci çözümdeki gibi her rakam için basamak değeri baştan hesaplanmaz.
  - Analiz (`'3' '0' '5'`, N = 3):

carpan	I	rakamlar[I]	sayi	toplam
1	3	`'5'`	5	5
10	2	`'0'`	0	5
100	1	`'3'`	3	305

- Algoritma 12: küçük harfli kelimeyi büyük harfe dönüştürme (ekrandan, 01:12:05)
  - Soru: "Küçük harflerden oluşan bir kelimeyi BÜYÜK HARFLERE dönüştüren algoritmanın akış diyagramını çiziniz." Örnek: `abc` → `ABC`.
  - N okunur, `I ← 1 || 1 || N` ile `kelime[I]` okunur.
  - Dönüşüm döngüsü: `buyuk[I] = kelime[I] + 'A' − 'a'`.
  - Son döngüde `buyuk[I]` tek tek yazdırılır.
  - Mantık: bir harfin küçük `'a'`'ya uzaklığı ne ise, büyük karşılığının büyük `'A'`'ya uzaklığı da odur. Örneğin `'b'`, `'a'`'dan 1 uzaktadır. Büyük karşılığı da `'A'`'dan 1 uzaktadır, yani `'B'`'dir.

  - Bu, birçok editör ve dildeki “uppercase” fonksiyonunun çalışma prensibidir.
- Algoritma 13: palindrom kelime (ekrandan, 01:53:40)
  - Soru: “Verilen bir kelimenin palindrom kelime olup olmadığını bulan algoritmanın akış diyagramını çiziniz.”
  - Palindrom: düz ve tersten okunuşu aynı olan kelime. Örnekler: `ses`, `kabak`, `baab`. Harf sayısı tek de olabilir, çift de.
  - Fikir: Her adımda soldan bir karakter ve sağdan bir karakter alınıp karşılaştırılır. Sol indis bir artırılır, sağ indis bir azaltılır. İç içe iki döngüye gerek yoktur.
  - Döngü türü: ilk farklılığın nerede çıkacağı belli olmadığı için while kullanılır.
  - Birinci çözüm (`palindrom` değişkeniyle):
    * `palindrom = 1`, `I = 1`.
    * While: `I < (N DIV 2 + 1) AND palindrom = 1`.
    * İçeride `kelime[I] ≠ kelime[N−I+1]` ise `palindrom = 0`. Sonra `I = I + 1`.
    * Döngüden sonra `palindrom = 1` ise “palindrom”, değilse “palindrom değil” yazdırılır.
  - İkinci çözüm (iki indisle):
    * `I = 1`, `J = N`.
    * While: `I < J AND kelime[I] = kelime[J]`. İçeride `I = I + 1`, `J = J − 1`.
    * Döngüden sonra `I >= J` ise “palindrome”, değilse “not palindrome” yazdırılır.
- Dersin son bölümü bilgisayar mühendisliği bölümünün oryantasyon eğitimine ayrılmıştır (02:07:01 sonrası). Bu kısım ders konusu olmadığı için özetlenmedi.

Hocanın Özellikle Vurguladığı Kısımlar

- ASCII tablosundaki sayısal değerleri (ör. `'0'` için 48) kullanmayın. Bu, değerleri ezberlemek zorunda olmanız demektir. Karakterin değerine ihtiyacınız varsa sabitin kendisini (`'0'`, `'a'`, `'A'`, `'z'`, `'Z'`) kullanın.
- Atama her zaman sağdan sola yapılır. `rakamlar[I] − '0' = sayi` gibi bir yazım asla kabul edilmez. Hoca böyle bir ifade gördüğünde sorunun üzerini çizer. Doğrusu `sayi = rakamlar[I] − '0'`.
- Bir çözümün doğru olup olmadığına analiz yaparak karar verin. “Bence çalışmaz” demek yetmez, değişkenleri adım adım takip edin.
- Döngüler büyükten küçüğe de kurulabilir. Yeter ki başlangıç, bitiş ve artım/azaltım doğru verilsin.
- Yanlış cevap vermek ayıp değildir. Derse katılın ve fikir yürütün. Yanlışın tartışılması doğruyu öğrenmeyi kolaylaştırır.
- Bir soruyu internetten bakarak çözdüyseniz bunu belirtin. Önemli olan kendi başınıza düşünebilmektir.
- Okuma, işlem ve yazdırma tek döngüde yapılabilir. Derste anlaşılır olması için bunlar ayrı döngülerde yapılıyor. Elde edilen yeni bilgi genelde ya aynı dizinin üzerine ya da yeni bir dizide saklanır, sonra yazdırılır.
- Rakam karakterlerinin bulunduğu dizinin üzerine sayısal değer yazılırsa karakterin kendisi kaybolur. Bu yüzden ayrı bir `sayi` değişkeni kullanılır.
- Büyük harf dönüşümünde küçük hâle ihtiyaç yoksa sonuç `kelime[I]`’nın üzerine de yazılabilir.
- If’in true kolu boş bırakılamaz. Palindrom çözümünde “eşitse `palindrom = 1`” yazmak yerine koşul ters kurulmalıdır: `kelime[I] ≠ kelime[N−I+1]` ise `palindrom = 0`. False kolu boş kalır.
- While’dan çıkınca neden çıkıldığı bilinmez. Gerçekten farklı bir çift mi bulundu, yoksa bütün çiftler mi kontrol edildi? Bunu anlamak için döngüden sonra bir if gerekir.
- For ile bütün çiftleri kontrol eden çözüm doğru çalışır ama verimli değildir. İlk farklılıkta çıkmak gerekir. Kelime uzadıkça gereksiz kontrol sayısı artar.
- Sınır değerleri her zaman yanıltabilir. Bir çözüm iki örnekte doğru, üçüncüsünde yanlış çalışabilir. Tek ve çift uzunluklu kelimeleri, eşleşen ve eşleşmeyen örnekleri ayrı ayrı deneyin.

Kısa Tekrar Notları

- Rakam karakterinin değeri: `c − '0'`.

- Küçük harfi büyüğe çevirme: `c + 'A' - 'a'`.
- C'de karakter dizisi null karakteriyle (0) biter.
- Rakamlardan sayı: sağdan sola, `toplam += (rakamlar[I] - '0') * carpan`, `carpan *= 10`.
- Tek döngülü çözüm, her rakam için çarpanı baştan hesaplayan çözümden daha verimlidir.
- Palindrom 1: `I < N DIV 2 + 1 AND palindrom = 1`, eşit değilse `palindrom = 0`.
- Palindrom 2: `I < J AND kelime[I] = kelime[J]` sürece `I++`, `J--`. Sonda `I >= J` ise palindrom.
- Son if'te `I <= J` yazılırsa çift uzunluklu `baab` yanlış sonuç verir. Doğrusu `I >= J`.
- True kolu boş if olmaz. Koşulu tersine çevirin.

Detaylı Açıklamalar

Ders, önceki derste verilen "rakam karakterlerinden sayı oluşturma" sorusunun çözümüyle başlar. Bir öğrencinin iki iç içe döngülü çözümü analiz edilir ve doğru çalıştığı görülür. Ardından tek döngülü, daha verimli bir çözüm birlikte yazılır. Sonra küçük harfli kelimeyi büyük harfe çeviren algoritma ve palindrom kelime algoritması çözülür. Hoca, gelecek hafta C programlama diline geçileceğini ve o dersten sonra şimdiye kadar çizilen akış diyagramlarının kodunun yazılabileceğini söyler. Dersin son bölümü bölüm oryantasyonuna ayrılmıştır.

Rakamlardan sayı: iki çözümün karşılaştırılması

Birinci çözümde her rakam için çarpan 1'den başlatılır ve iç döngüde $N - I$ kez 10 ile çarpılır. `'3' '0' '5'` için:

- I = 1: iç döngü 2 kez döner, çarpan 100 olur. `toplam = 0 + 3 * 100 = 300`.
- I = 2: iç döngü 1 kez döner, çarpan 10 olur. `toplam = 300 + 0 * 10 = 300`.
- I = 3: iç döngüye hiç girilmez ($J \leq 0$), çarpan 1 kalır. `toplam = 300 + 5 * 1 = 305`.

İç döngünün bitişi önce `N-1` olarak yazıldı, sonra bir öğrencinin uyarısıyla `N-I` olarak düzeltildi (`N-1`'in söyleyişten mi yazımdan mı geldiği altyazıdan net değil). `N-1` ile her rakam aynı çarpanla çarpılırdı ve sonuç yanlış olurdu.

İkinci çözümde dizi sağdan sola dolaşılır. Birler basamağı ilk sırada işlenir. Çarpan her turda 10 ile çarpılarak 1, 10, 100 değerlerini alır. `'3' '0' '5'` örneğinde 10 ile çarpma sayısı iki çözümde de 3'tür. Genel olarak birinci çözümde $N(N-1)/2$, ikincide N çarpma yapılır, yani rakam sayısı arttıkça ikinci çözüm daha verimlidir. Bir öğrenci analiz yapmadan "çalışmaz" dedi. Hoca tabloyla adım adım takip ederek çözümün 305 sonucunu verdiğini gösterdi.

Analiz tablosunda bir değişkenin yeni değeri bir sonraki satıra kaydırılarak da yazılabilir. Derste yer kazanmak için değerler aynı satıra yazıldı. İki gösterim de kabul edilir.

Büyük harfe dönüştürme neden çalışır?

`'a'` ile `'z'` arası küçük harfler ve `'A'` ile `'Z'` arası büyük harfler ASCII tablosunda ardışık sıralanır. Bu yüzden:

$$\text{buyuk} = \text{kelime} - \text{'a'} + \text{'A'}$$

Önce harfin `'a'`'ya uzaklığı bulunur. Bu uzaklık `'A'`'ya eklenince büyük karşılığı elde edilir. Akış diyagramında sonuç sayısal değer gibi görünür. C'de karakter tipinde bir dizi tanımlanıp karakter formatıyla yazdırıldığında ekranda `A`, `B`, `C` görünür. `%s` formatıyla karakter grubunun tamamı tek seferde de yazdırılabilir.

Palindrom çözümünde sınır değerleri

İkinci çözümde döngüden sonraki if koşulu tartışıldı.

- Önce while koşulu yanlışlıkla `I > J` yazıldı ve `I < J` olarak düzeltildi.

- `kabak` (N = 5) için: (1,5) eşit, (2,4) eşit, sonra I = J = 3 olur ve döngü biter. If'te `I < J` sorulursa sonuç "palindrom değil" çıkar, yani yanlış olur. `I <= J` yapıldığında bu örnek doğru çalışır. (If koşulu altyazıda önce "küçük eşit", kabak analizinde "küçük" olarak geçer.)
- `baab` (N = 4) için: (1,4) eşit, (2,3) eşit, sonra I = 3, J = 2 olur. `I <= J` sorulursa yine yanlış sonuç çıkar.
- Doğru koşul `I >= J`'dir. Döngü yalnızca bütün çiftler eşleştiğinde I ile J buluşur veya birbirini geçer. Farklı bir çift bulunursa döngü $I < J$ iken biter.
- Kontrol örnekleri: `abcd` için (1,4) farklı, döngü I = 1 < J = 4 iken biter, sonuç "not palindrome". `ses` için (1,3) eşit, sonra I = J = 2 olur, sonuç "palindrome". Bir öğrencinin önerdiği `abca` için (1,4) eşit, (2,3) farklı, döngü I = 2 < J = 3 iken biter, sonuç "not palindrome". Bu da doğrudur.

Birinci çözümde ise döngü kelimenin ortasına kadar gider: `I < N DIV 2 + 1`. `kabak` için 5 DIV $2+1=3$ olduğu için I = 1 ve 2 kontrol edilir. Ortadaki harfin kendisiyle karşılaştırılmasına gerek yoktur. `baab` için 4 DIV $2+1=3$ olduğu için yine I = 1 ve 2 kontrol edilir.

Hoca iki çözümü şöyle karşılaştırdı:

- Birinci çözüm, `palindrom` değişkenini 1'e set edip 1 olduğu sürece devam eder. Koşul bozulursa değişkeni değiştirir. Döngüde iki kontrol vardır (while ve if) ve fazladan `N DIV 2 + 1` hesaplanır. Öğrenmenin ilk aşamasında daha az karışık olduğu için bu tarz çözümler kabul edilebilir.
- İkinci çözümde ayrı bir değişken yoktur. Koşul doğrudan while'a yazılır ve kelimeye iki uçtan yaklaşılır. Daha şıktır, ama sınır değerleri çok dikkatli seçilmelidir. Zamanla bu tarz çözümlere geçilmesi beklenir.

Ders sonrasında yapılan kontrolde iki palindrom çözümü de 1 ile 7 harf arasındaki bütün `a/b` kelimeleri için doğru sonuç verdi. Rakamlardan sayı oluşturan çözüm de `'3' '0' '5'` için 305 sonucunu verdi.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.